Leakage-Resilient Multi-Party Computation: Protecting the Evaluator in Circuits Garbling

Francesco Berti
EnICS, Selecsys Labs
Bar-Ilan University
Ramat-Gan, Israel
francesco.berti@biu.ac.il

Itamar Levi
EnICS, Selecsys Labs
Bar-Ilan University
Ramat-Gan, Israel
itamar.levi@biu.ac.il

***Abstract*—Garbling schemes allow two parties to compute a joint function on private inputs without revealing them. Yet, a semi-honest garbler might exploit hardware/software side-channel leakages from the evaluator. An alarming threat with no concrete solution yet. Using the homomorphic properties of ElGamal encryption, we can prevent such leakage-based attacks.**

***Index Terms*—cryptography, MPC, garbling, leakage**

I. Introduction

Cryptography addresses many challenges in today's digital world. A notable example is *Multi-Party Computation* (*MPC*), where multiple parties, each holding a private input x_i, jointly compute a function $f(x_1, \ldots, x_n)$ without revealing their inputs. Each party learns only $f(x_1, \ldots, x_n)$ [1]–[3].

In the two-party case, *garbled circuits* are commonly used: one party (the *garbler*) constructs a circuit encoding $f(x_1, \cdot)$ using its input x_1 and sends it to the other party (the *evaluator*), who evaluates it on its input x_2 to obtain $f(x_1, x_2)$ [1], [2]. Garbling proceeds gate by gate: each wire has two labels, X_i^0 and X_i^1, for values 0 and 1. For a gate with inputs A, B, output C, and functionality g the garbler encodes $X_C^{g(j,j')}$ using X_A^j and $X_B^{j'}$, shuffles the encodings, and sends them to the evaluator. The evaluator receives, via *Oblivious Transfer*, the labels for its inputs, and evaluates each gate by decoding using its labels, since decoding with mismatched labels yields no output [1], [3]. Many works improve the efficiency of such schemes [2], [4], [5].

These protocols execute on physical devices, which may leak side-channel information such as timing, power consumption, or electromagnetic emissions [6], [7]. Leakage can reveal secret keys or intermediate values and enable distinguishing attacks [8], [9]. This is particularly relevant when a semi-honest garbler attacks the evaluator with leakage, since he knows all the values used to generate the garbled data, making this the simplest side-channel attack [8], [10]–[12], see Fig. 2.

Our contributions: We define security for garbling schemes under leakage when the garbler is semi-honestly corrupted and propose a construction that mitigates comparison attacks. Our approach leverages the homomorphic property of the El Gamal encryption scheme: the garbler encodes output labels with El Gamal, while the evaluator randomizes each ciphertext before decryption, preventing the garbler from reproducing its leakage profile.

I. Levi and F. Berti funded by Israel Science Foundation, ISF grant 2569/21.

II. Preliminaries

We denote by $x \xleftarrow{\$} \mathcal{X}$ sampling x uniformly from $\mathcal{X}$. A t-adversary is an algorithm running in time at most t.

El Gamal Encryption: Let $\mathbb{G}$ be a cyclic group of order q with generator g. *Key generation* samples $x \xleftarrow{\$} \mathbb{Z}_q$, outputs the *public key* $h = g^x$ and the *secret key* x. To *encrypt* $m \in \mathbb{G}$, choose $r \xleftarrow{\$} \mathbb{Z}_q$ and compute $c_1 = g^r$, $c_2 = h^r m$, returning $c = (c_1, c_2)$. *Decryption* with x recovers $m = c_2/c_1^x$. If the discrete logarithm problem is hard in $\mathbb{G}$, El Gamal is secure [13]. The scheme is *multiplicatively homomorphic*: for ciphertexts c, d encrypting m, m', $cd = (c_1 d_1, c_2 d_2)$ encrypts mm' since $(c_1 d_1)^x = c_1^x d_1^x$.

Garbling Schemes: A *garbling scheme* $\mathsf{Gb} = (\mathsf{GBC}, \mathsf{Enco}, \mathsf{Eval}, \mathsf{Deco})$ consists of: 1) *Circuit garbling* from the security parameter n and a circuit $\mathbf{C}$, $\mathsf{GBC}(n, \mathbf{C})$ outputs the *garbled circuit* F, the *encoding function* E, and the *decoding function* D. 2) *Input encoding* $\mathsf{Enco}(E, x)$ garbles the input x into the *garbled input* X. 3) *Evaluation algorithm* $\mathsf{Eval}(F, X)$ outputs the *garbled output* Y. 4) *Output decoding* $\mathsf{Deco}(D, Y)$ outputs y or "invalid". We require *correctness*: $\mathsf{Deco}(D, \mathsf{Eval}(F, \mathsf{Enco}(E, x))) = \mathbf{C}(x)$, except with negligible probability. Security requires that the evaluator learns no more about the garbler's input than $\mathbf{C}(x)$ [2].

Yao's Scheme: Each wire i has two labels X_i^0, X_i^1 for bits 0 and 1. For a gate with functionality g and input wire A, B, output wire C, the garbler computes $C^{j,j'} := \mathsf{Enc}_{X_A^j}(\mathsf{Enc}_{X_B^{j'}}(X_C^{g(j,j')}))$, sends the four ciphertexts (shuffled), and the evaluator decrypts them using its input labels received via Oblivious Transfer. If the encryption is CPA-secure and $\mathsf{Dec}_{X_A^{j\oplus 1}}(\mathsf{Enc}_{X_A^j}(m))$ yields "invalid," the scheme is secure [1], [3].

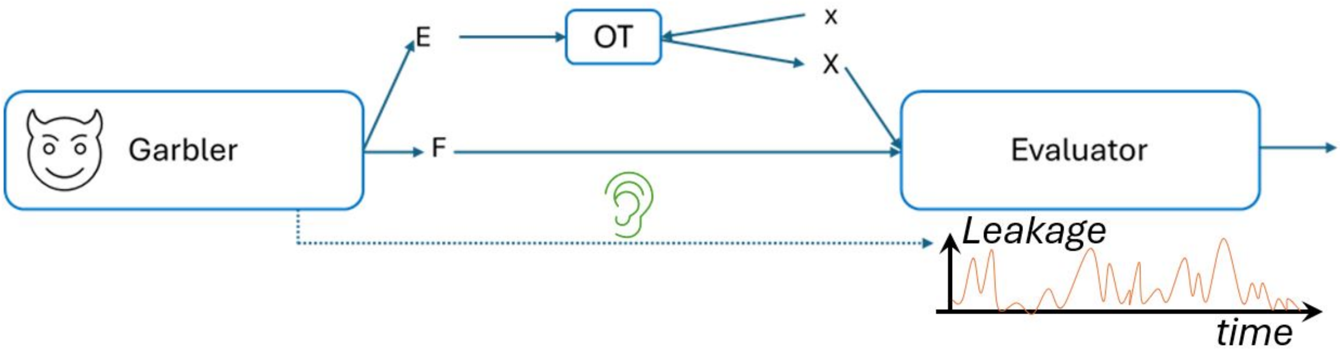

Fig. 1. Our security scenario

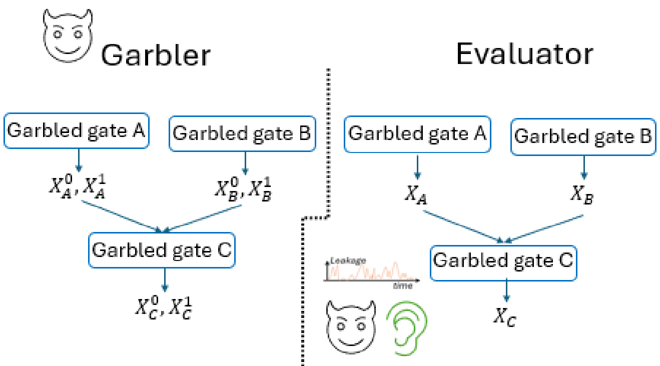

Fig. 2. The security scenario looking at garbled gates.

III. Security definition for the evaluator in the presence of leakage

Notation. We denote by L the *leakage function* capturing the information obtained by the adversary.

In the absence of leakage, the garbler learns nothing from the evaluator, who only outputs the final result. However, if the garbler has physical access to the evaluator's device, it may exploit leakage. We thus adapt the indistinguishability notion to this setting.

Definition 1 (Indistinguishability with evaluator leakage): Let $\mathsf{Gb} = (\mathsf{GBC}, \mathsf{Enco}, \mathsf{Eval}, \mathsf{Deco})$ be a garbling scheme, and let $\mathsf{L}_{\mathsf{Eval}}$ be the leakage function associated with the evaluator's execution of Eval. We say that Gb provides (t, ϵ)-*indistinguishability with leakage for the evaluator* if for every t-adversary $\mathsf{A} = (\mathsf{A}_1, \mathsf{A}_2)$ and every efficient circuit $\mathbf{C}$,

$$\begin{aligned} |\Pr[b = b' \mid b' \leftarrow \mathsf{A}_2(\ell, \mathsf{st}),\ \ell = \mathsf{L}_{\mathsf{Eval}(F, X_b)}, \\ X_b = \mathsf{Enco}(E, x_b),\ (x_0, x_1, \mathsf{st}) \leftarrow \mathsf{A}_1(F, E, D), \\ (F, E, D) \leftarrow \mathsf{GBC}(n, \mathbf{C}),\ b \xleftarrow{\$} \{0,1\}] - 1/2| \leq \epsilon(n). \end{aligned}$$

IV. Our Garbling Scheme

Our construction builds upon Yao's garbling scheme, replacing the encryption primitive with a variant of the El Gamal encryption scheme. Given public parameters $(\mathbb{G}, g, q, h)$, encryption is defined as

$$\mathsf{Enc}_x(m) := (g^{r_1}, h^{r_1} m, g^{r_2}, h^{r_2} m),$$

where $r_1, r_2 \xleftarrow{\$} \mathbb{Z}_q$. That is, each message m is doubly encrypted, ensuring that for any $x' \neq x$ $h^{r_1} m / g^{r_1 x'} \neq h^{r_2} m / g^{r_2 x'}$. We instantiate $\mathbb{G}$ as the cyclic group $\mathbb{Z}_p^*$ for a sufficiently large prime p, allowing encrypted messages to serve as new secret keys (exponents for subsequent h values).

During evaluation, the evaluator picks two random messages m_1 and m_2, it encrypts them as $(g^{r_3}, h^{r_3} m_1)$ and $(g^{r_4}, h^{r_4} m_2)$, then decrypts

$$(g^{r_1} g^{r_3}, h^{r_1} m h^{r_3} m_1) \quad \text{and} \quad (g^{r_2} g^{r_4}, h^{r_2} m h^{r_4} m_2)$$

using its private key x. It finally checks whether the ratio mm_1/mm_2 equals m_1/m_2. We employ El Gamal solely for its multiplicatively homomorphic property, not as a public-key scheme, since both parties know the secret key.

This construction prevents the garbler from precisely profiling the evaluator's leakage, as the ciphertexts decrypted using x appear random and unpredictable to the garbler.

Finally, we explain how to apply our technique to compute ciphertexts of the form $\mathsf{Enc}_{X_A^j}(\mathsf{Enc}_{X_B^{j'}}(X_C^{g(j,j')}))$ in Yao's construction. There are two options: 1) Encrypt $X_C^{g(j,j')}$ using $X_B^{j'}$ as described above, obtaining a 4-component ciphertext, and then encrypt each of these components using X_A^j; or 2) Encrypt $X_C^{g(j,j')}$ directly under the combined key $X_A^j + X_B^{j'}$. Both approaches preserve correctness and allow us to integrate our leakage-resilient encryption layer into Yao's garbling framework.

V. Conclusions and Prospects

We presented the first formal security definition for a garbling scheme resilient to leakage attacks from a semi-honest garbler. We also proposed a concrete (though computationally heavy) construction achieving this property. Future work should focus on optimizing efficiency and exploring alternative primitives that preserve leakage resilience.

REFERENCES

[1] A. C. Yao, "How to generate and exchange secrets (extended abstract)," in *FOCS*, 1986.

[2] C. Hazay and Y. Lindell, *Efficient Secure Two-Party Protocols - Techniques and Constructions*. Information Security and Cryptography, Springer, 2010.

[3] Y. Lindell and B. Pinkas, "A proof of yao's protocol for secure two-party computation," *Electron. Colloquium Comput. Complex.*, 2004.

[4] V. Kolesnikov and T. Schneider, "Improved garbled circuit: Free XOR gates and applications," in *ICALP*, 2008.

[5] B. Pinkas, T. Schneider, N. P. Smart, and S. C. Williams, "Secure two-party computation is practical," in *ASIACRYPT*, 2009.

[6] P. C. Kocher, J. Jaffe, and B. Jun, "Differential power analysis," in *CRYPTO*, 1999.

[7] F. Berti, S. Bhasin, J. Breier, X. Hou, R. Poussier, F. Standaert, and B. Udvarhelyi, "A finer-grain analysis of the leakage (non) resilience of OCB," *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, vol. 2022.

[8] C. Guo, O. Pereira, T. Peters, and F. Standaert, "Authenticated encryption with nonce misuse and physical leakage: Definitions, separation results and first construction - (extended abstract)," in *LATINCRYPT*, 2019.

[9] F. Berti, O. Pereira, T. Peters, and F. Standaert, "On leakage-resilient authenticated encryption with decryption leakages," *IACR Trans. Symmetric Cryptol.*, vol. 2017, no. 3.

[10] I. Levi and C. Hazay, "Garbled circuits from an sca perspective: Free xor can be quite expensive...," *IACR Transactions on Cryptographic Hardware and Embedded Systems*, pp. 54–79, 2023.

[11] R. Li, Y. Sun, C. Guo, F. Standaert, W. Wang, and X. Wang, "Leakage-resilient circuit garbling," in *CCS*, 2024.

[12] F. Berti, C. Hazay, and I. Levi, "LR-OT: leakage-resilient oblivious transfer," *Cryptogr. Commun.*, 2025.

[13] J. Katz and Y. Lindell, *Introduction to Modern Cryptography, Second Edition*. CRC Press, 2014.